

Method Overloading in Java

What is Method Overloading?

Method Overloading allows a class to have multiple methods with the **same name** but **different parameters**. Java decides which method to call based on the arguments passed during compilation. This is known as **Compile-Time Polymorphism** or **Static Polymorphism**.

Rules of Method Overloading

- ✓ Same Method Name
 - ✓ Different Number of Parameters
 - ✓ Different Data Types of Parameters
 - ✓ Different Order of Parameters
 - ✓ Return Type may be same or different
 - ✗ Changing only Return Type is NOT Overloading
-

Program 1: Overloading by Data Type

```
class Adder {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
  
    static double add(double a, double b) {  
        return a + b;  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println(Adder.add(11,11));  
        System.out.println(Adder.add(12.3,12.6));  
    }  
}
```

Output

22
24.9

This example is similar to the reference where Java chooses the correct method based on parameter types.

Program 2: Overloading by Number of Parameters

```
class Calculator {  
  
    void sum(int a, int b) {  
        System.out.println(a + b);  
    }  
  
    void sum(int a, int b, int c) {  
        System.out.println(a + b + c);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Calculator obj = new Calculator();  
  
        obj.sum(10,20);  
        obj.sum(10,20,30);  
    }  
}
```

Output

30
60

Program 3: Overloading by Parameter Order

```
class Display {  
  
    void show(int id, String name) {  
        System.out.println(id + " " + name);  
    }  
  
    void show(String name, int id) {  
        System.out.println(name + " " + id);  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Display d = new Display();  
  
        d.show(101,"Rahul");  
        d.show("Ravi",102);  
    }  
}
```

Output

```
101 Rahul  
Ravi 102
```

Program 4: Constructor Overloading

```
class Student {  
  
    Student() {
```

```
        System.out.println("Default Constructor");
    }

    Student(String name) {
        System.out.println("Student Name : " + name);
    }
}

public class Main {

    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student("Rahul");
    }
}
```

Output

Default Constructor
Student Name : Rahul

Method Overriding in Java

What is Method Overriding?

Method Overriding allows a child class to provide its own implementation of a method already present in the parent class. The method name and parameters must remain the same. This is called **Runtime Polymorphism** or **Dynamic Polymorphism**.

Rules of Method Overriding

- ✓ Inheritance Required
- ✓ Same Method Name
- ✓ Same Parameters
- ✓ Same Return Type (or Covariant Return Type)
- ✓ Runtime Decision
- ✗ Static Methods Cannot Be Overridden
- ✗ Final Methods Cannot Be Overridden

Program 1: Basic Method Overriding

```
class Vehicle {  
  
    void run() {  
        System.out.println("Vehicle is running");  
    }  
}  
  
class Bike extends Vehicle {  
  
    void run() {  
        System.out.println("Bike is running safely");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Bike obj = new Bike();  
  
        obj.run();  
    }  
}
```

Output

Bike is running safely

This example follows the same concept shown in the reference material.

Program 2: Runtime Polymorphism

```
class Animal {  
  
    void sound() {
```

```
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {

    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {

    public static void main(String[] args) {

        Animal a = new Dog();

        a.sound();
    }
}
```

Output

Dog barks

Explanation

Even though the reference variable is of type `Animal`, Java executes the `Dog` version at runtime.

Program 3: Bank Interest Example

```
class Bank {

    int getRate() {
        return 0;
    }
}

class SBI extends Bank {
```

```
    int getRate() {
        return 7;
    }
}

class HDFC extends Bank {

    int getRate() {
        return 8;
    }
}

public class Main {

    public static void main(String[] args) {

        SBI s = new SBI();
        HDFC h = new HDFC();

        System.out.println("SBI Rate : " + s.getRate());
        System.out.println("HDFC Rate : " + h.getRate());
    }
}
```

Output

SBI Rate : 7
HDFC Rate : 8

Program 4: Shape Example

```
class Shape {

    void draw() {
        System.out.println("Drawing Shape");
    }
}

class Circle extends Shape {
```

```
void draw() {  
    System.out.println("Drawing Circle");  
}  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Shape s = new Circle();  
  
        s.draw();  
    }  
}
```

Output

Drawing Circle

Method Overloading vs Method Overriding

Feature	Method Overloading	Method Overriding
Class Relationship	Same Class	Parent & Child Class
Method Name	Same	Same
Parameters	Different	Same
Return Type	Can be Different	Must be Same/Covariant
Inheritance	Not Required	Required
Polymorphism Type	Compile-Time (Static)	Runtime (Dynamic)
Binding	Early Binding	Late Binding
Static Methods	Can be Overloaded	Cannot be Overridden
Decision Taken	During Compilation	During Runtime